

TypeScript for a Java and Python engineer

The win: read every line of epic 1 as it lands. Eight ideas, each one a file you run, then break on purpose.

How to use this page. Everything here lives in `drills/lesson-0/` of the repo. Node 24 runs TypeScript directly, so there is nothing to build:

```
cd drills/lesson-0
node 01-values.ts          # run an exercise
sh check.sh                # prove all 17 files behave as this page says
```

For each section: predict the output, run it, then open the `.broken.ts` twin and predict *how* it fails before you look.

- 1 · Values, inference, one number type
- 2 · Object types and structural typing
- 3 · Unions, literals, narrowing
- 4 · Functions and generics
- 5 · Types vanish at runtime
- 6 · Two kinds of nothing
- 7 · Modules and the `.ts` rule
- 8 · Promises and the single thread
- 9 · Reading this repo's config

Recall, then test yourself

1 · Values, inference, one number type

`const` is a binding that never changes; `let` may. You rarely write a type on a local variable: the compiler infers it. Notice that a `const` string gets the *literal* type `"api"`, not `string`. That tiny fact powers section 3.

Typescript

Java

Python

`const / let`

`final var / var`

no equivalent; everything rebinds

one `number` (64-bit float) + `bigint`

`int long float double`
`BigInteger`

`int` (unbounded) + `float`

`1 / 2 === 0.5`

`1 / 2 == 0`

`1 / 2 == 0.5`

RUN IT · `node 01-values.ts`

```
const service = "api"; // inferred as the literal type "api", not string: const can never change
let attempts = 0; //      inferred as number: let can change, so TS widens to the general type
attempts += 1;

// There is no int/long/float/double. One `number` (a 64-bit float), plus `bigint` for big integers.
const half = 1 / 2; //      0.5, not 0 (Java's int division would give 0)
const big = 2n ** 64n; //      bigint literal: note the n
const unsafe = 2 ** 53 + 1; //      numbers lose integer precision above 2^53

console.log({ service, attempts, half, big, unsafe, safe: Number.isSafeInteger(unsafe) });

// Template strings replace String.format / f-strings.
console.log(`${service} took ${attempts} attempt(s)`);

// Annotations are optional. Write them on function boundaries, let inference do the rest.
const port: number = 8080;
console.log(typeof port); // "number" - typeof is a RUNTIME operator and only knows 8 JavaScript types
```

WHAT IT PRINTED

```
{
  service: 'api',
  attempts: 1,
  half: 0.5,
  big: 18446744073709551616n,
  unsafe: 9007199254740992,
  safe: false
}
api took 1 attempt(s)
number
```

Why it matters here: document sequence numbers come back from Postgres as `bigint`, and the driver hands them to us as *strings*. Adding 1 to that string gives `"101"` ... then `"1011"`. We convert once, at the database boundary.

BREAK IT • 01-values.broken.ts

```
// A const's literal type, and a let's widened type, are both enforced.
let attempts = 0;
attempts = "three"; // Type 'string' is not assignable to type 'number'.
console.log(attempts);
```

Fails with TS2322. Inference is not "dynamic typing". Once `attempts` is inferred as `number` it stays one.

2 • Object types and structural typing

This is the biggest mental shift from Java. Java is *nominal*: a class is a `HasName` only if it says `implements HasName`. TypeScript is *structural*: anything with a `name: string` is a `HasName`. Python calls this duck typing; TypeScript checks the duck at compile time.

TYPESCRIPT	JAVA	PYTHON
<pre>type Org = { id: string }</pre>	<pre>record Org(String id)</pre>	<pre>@dataclass / TypedDict</pre>
fits if the shape fits	fits if it declares <code>implements</code>	fits if it quacks (checked at runtime, or by <code>mypy Protocol</code>)
<pre>plan?: "free" optional field</pre>	<pre>Optional<Plan> or nullable</pre>	<pre>plan: str None = None</pre>

RUN IT • node 02-objects.ts

```
// Two ways to name an object shape. Rule of thumb in this repo: `type` by default,
// `interface` when something is meant to be extended or implemented.
type Org = { readonly id: string; name: string; plan?: "free" | "team" };
interface HasName { name: string }

const acme: Org = { id: "org_1", name: "Acme" }; // plan is optional, so it may be absent

// Structural typing: a value fits a type if it has the right SHAPE.
// Org never declares "implements HasName" (Java would require that). It just has a name.
function greet(x: HasName): string {
  return `hello ${x.name}`;
}
console.log(greet(acme));
console.log(greet({ name: "anything with a name" }));
```

```
// A class is not special either: this object literal is a valid Point.
class Point {
  x: number;
  y: number;
  constructor(x: number, y: number) {
    this.x = x;
    this.y = y;
  }
}

const p: Point = { x: 1, y: 2 }; // no `new`, no inheritance, still a Point: only the
shape matters
console.log(p, p instanceof Point); // instanceof is a RUNTIME check and says false

// readonly is compile-time only; it is erased like every other type.
console.log(Object.isFrozen(acme)); // false
```

WHAT IT PRINTED

```
hello Acme
hello anything with a name
{ x: 1, y: 2 } false
false
```

Read the last two outputs again. A plain object passed as a `Point`, yet `instanceof Point` is `false`, and `readonly` froze nothing. The type checker and the running program are two separate worlds. Section 5 is about the gap between them.

BREAK IT • 02-objects.broken.ts

```
// Structural typing is strict about object LITERALS: an unknown extra key is almost
always a typo.
type Org = { id: string; name: string };
const acme: Org = { id: "org_1", name: "Acme", nmae: "typo" };
console.log(acme);
```

Fails with TS2353. Structural does not mean sloppy: on a fresh object literal, an unknown key is treated as a typo.

3 • Unions, literals, narrowing

A value can be *one of several* types, and a type can be a single literal like `"running"`. Put a literal tag on each member and you get a **discriminated union**: the compiler follows your `switch` and knows, inside each case, exactly which fields exist. Our whole document protocol (four ops) is one of these.

TYPESCRIPT

JAVA

PYTHON

"queued" | "running"

enum RunStatus

Literal["queued",
"running"]

discriminated union + never
check

sealed interface + pattern
switch (Java 21)

match + assert_never

unknown (must narrow) vs any
(checker off)

Object

object vs Any

RUN IT · node 03-unions.ts

```
// This is the feature Java and Python do not have, and the one this codebase leans on
most.

// A literal union replaces most enums: no runtime object, just checked strings.
type RunStatus = "queued" | "running" | "succeeded" | "failed";

// A DISCRIMINATED union: every member carries a literal tag (`type`). Our four document
ops look like this.
type Op =
  | { type: "add_node"; nodeId: string; parentId: string }
  | { type: "remove_node"; nodeId: string }
  | { type: "set_prop"; nodeId: string; key: string; value: unknown };

function describe(op: Op): string {
  // Narrowing: inside each case the compiler KNOWS which member it is.
  switch (op.type) {
    case "add_node":
      return `add ${op.nodeId} under ${op.parentId}`; // parentId exists only on this
member
    case "remove_node":
      return `remove ${op.nodeId}`;
    case "set_prop":
      return `set ${op.key} on ${op.nodeId}`;
    default: {
      // Exhaustiveness: if someone adds a 5th op and forgets a case, `op` is no longer
`never`
      // here and this line stops compiling. (Java 21 sealed interfaces + switch do the
same.)
      const unreachable: never = op;
      return unreachable;
    }
  }
}

const status: RunStatus = "running";
console.log(status, "|", describe({ type: "set_prop", nodeId: "n1", key: "label", value:
"Pay" }));

// `unknown` is the safe "anything": you must narrow before using it. `any` switches the
checker off.
```

```
function lengthOf(x: unknown): number {
  if (typeof x === "string") return x.length; // narrowed to string
  if (Array.isArray(x)) return x.length; // narrowed to any[]
  return 0;
}
console.log(lengthOf("four"), lengthOf([1, 2]), lengthOf(42));
```

WHAT IT PRINTED

```
running | set label on n1
4 2 0
```

Why no `enum`? A TypeScript `enum` generates real JavaScript, and Node runs our files by *deleting* types, not compiling them. Literal unions cost nothing at runtime. Section 5's broken file proves it.

BREAK IT • 03-unions.broken.ts

```
// Add a member to the union, forget the case: the `never` line stops compiling.
type Op = { type: "add_node" } | { type: "remove_node" } | { type: "move_node" };
function describe(op: Op): string {
  switch (op.type) {
    case "add_node":
      return "add";
    case "remove_node":
      return "remove";
    default: {
      const unreachable: never = op; // Type '{ type: "move_node"; }' is not
      assignable to type 'never'.
      return unreachable;
    }
  }
}
console.log(describe({ type: "move_node" }));
```

Fails with TS2322. Add a fifth op and forget a case: the `never` line refuses to compile. The compiler finds every place you must update.

4 • Functions and generics

Functions are ordinary values. Generics look like Java's and are erased like Java's, but you almost never write the type argument: it is inferred from the call. There is no overloading by signature; use optional parameters, defaults, or an object parameter (which doubles as named arguments).

TYPESCRIPT	JAVA	PYTHON
<code>(n: number) => n * 2</code>	<code>n -> n * 2</code>	<code>lambda n: n * 2</code>
<code><T extends { id: string }></code>	<code><T extends HasId></code>	<code>TypeVar("T", bound=HasId)</code>
<code>f({ title, orgId })</code>	builder / many overloads	keyword arguments

RUN IT · node 04-functions.ts

```
// Functions are values. Both forms are everywhere; arrows are the norm for callbacks.
function add(a: number, b: number): number {
  return a + b;
}
const double = (n: number): number => n * 2;

// Optional and default parameters (no overloading by signature like Java).
function page(items: string[], size = 2, from?: number): string[] {
  const start = from ?? 0;
  return items.slice(start, start + size);
}

// Generics: like Java's, erased at runtime like Java's, but inferred at the call site
// far more often.
function first<T>(xs: readonly T[]): T | undefined {
  return xs[0]; // T | undefined: an empty array has no first element, and TS makes you
// say so
}

// A generic constraint ('extends') = Java's bounded type parameter.
function byId<T extends { id: string }>(rows: T[]): Map<string, T> {
  return new Map(rows.map((r) => [r.id, r]));
}

console.log(add(2, 3), double(4), page(["a", "b", "c"]), page(["a", "b", "c"], 1, 2));
console.log(first([10, 20]), first([]));
console.log(byId([{ id: "o1", name: "Acme" }]).get("o1"));

// Object parameter + destructuring is how "named arguments" are done.
function createDoc({ title, orgId }: { title: string; orgId: string }): string {
  return `${orgId}/${title}`;
}
console.log(createDoc({ orgId: "org_1", title: "Checkout" }));
```

WHAT IT PRINTED

```
5 8 [ 'a', 'b' ] [ 'c' ]
10 undefined
{ id: 'o1', name: 'Acme' }
org_1/Checkout
```

BREAK IT • 04-functions.broken.ts

```
// noUncheckedIndexedAccess: xs[0] is `number | undefined`, so arithmetic on it is
an error until you check.
function firstPlusOne(xs: number[]): number {
  return xs[0] + 1; // Object is possibly 'undefined'.
}
console.log(firstPlusOne([]));
```

Fails with TS2532. With `noUncheckedIndexedAccess` on (it is, repo-wide), `xs[0]` is `number | undefined`. Java would throw `IndexOutOfBoundsException` at runtime; here you handle the empty case before the code compiles.

5 • Types vanish at runtime

If you keep one idea from this lesson, keep this one. Node 24 runs a `.ts` file by replacing every type annotation with spaces and executing what is left. Nothing checks types while the program runs. An `as` cast is a promise you make to the compiler, not a check. So a JSON body, a WebSocket message, a database row, an environment variable: each is `unknown` until *runtime code* proves otherwise.

TYPESCRIPT	JAVA	PYTHON
types erased; <code>as</code> never checks	generics erased, but casts are checked (<code>ClassCastException</code>)	hints ignored at runtime unless a library reads them
Zod schema → inferred type	Jackson + Bean Validation	pydantic model

RUN IT • node 05-erased.ts

```
// Node runs this .ts file by deleting the type syntax. Nothing checks types while it
runs.

type Health = { status: "ok"; service: string };

// 1. A cast (`as`) is a promise to the compiler, not a check. This "Health" is a lie,
and it runs fine.
const fromTheWire = JSON.parse('{"status":"down","service":42}') as Health;
console.log("the compiler believes this is Health:", fromTheWire);

// Java's Jackson would throw here; Python + pydantic would raise. Plain TypeScript does
nothing.
// So at every trust boundary (HTTP body, WebSocket message, DB row, env var) we must
check at RUNTIME.

// 2. A hand-written runtime check. `x is Health` is a TYPE PREDICATE: it tells the
```

```

compiler
// "if this returns true, treat x as Health from here on".
function isHealth(x: unknown): x is Health {
  return (
    typeof x === "object" && x !== null &&
    "status" in x && x.status === "ok" &&
    "service" in x && typeof x.service === "string" && x.service.length > 0
  );
}

const candidate: unknown = JSON.parse('{"status":"ok","service":"api"}');
if (isHealth(candidate)) console.log("checked for real:", candidate.service); // narrowed
to Health
console.log("the liar passes the check?", isHealth(fromTheWire)); // false

// 3. Writing these by hand does not scale, and the type and the check can drift apart.
//   Zod (packages/contracts) writes ONE schema and DERIVES the type from it:
//   const Health = z.object({ status: z.literal("ok"), service: z.string().min(1) });
//   type Health = z.infer<typeof Health>; // the type comes from the runtime check,
so they cannot drift
//   Health.parse(json) // throws with the failing field's name

```

WHAT IT PRINTED

```

the compiler believes this is Health: { status: 'down', service: 42 }
checked for real: api
the liar passes the check? false

```

This is why `packages/contracts` exists. Every HTTP body and WebSocket message is defined once as a Zod schema; the TypeScript type is *derived* from the schema with `z.infer`, so the check and the type cannot drift apart. It is already in the repo: open `packages/contracts/src/index.ts`.

BREAK IT • 05-erased.broken.ts

```

// Node can only DELETE type syntax. `enum` generates real code, so Node refuses to
run it.
// (That is why this repo uses literal unions and sets `erasableSyntaxOnly`.)
enum RunStatus { Queued, Running }
console.log(RunStatus.Queued);

```

Fails with `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`. This one fails when you *run* it, not when you compile it. Node can only delete syntax; an `enum` needs code generated, so Node refuses. The same goes for `namespace`, constructor parameter properties and decorators. Our `tsconfig` sets `erasableSyntaxOnly` so the compiler rejects them first.

6 · Two kinds of nothing

JavaScript has `undefined` ("never set") and `null` ("deliberately empty"). With `strict` on, neither is a member of any type unless you say so: a `string` can never be null, unlike every Java reference. Two operators do most of the work: `?.` and `??`.

TYPESCRIPT	JAVA	PYTHON
<code>null</code> and <code>undefined</code>	<code>null</code>	<code>None</code>
<code>a?.b</code>	<code>Optional.map</code>	no operator
<code>a ?? d</code> (only null/undefined)	<code>Optional.orElse</code>	<code>a if a is not None else d</code> (not <code>a or d</code>)

RUN IT · `node 06-nothing.ts`

```
// Java has null. Python has None. JavaScript has BOTH null and undefined.
//   undefined = "was never set" (missing property, missing argument, array hole)
//   null      = "deliberately empty" (SQL NULL arrives as null; JSON has null but no
//   undefined)

type Doc = { title: string; description?: string; archivedAt: Date | null };
const doc: Doc = { title: "Checkout", archivedAt: null };

console.log(doc.description, doc.archivedAt); // undefined null

// ?. optional chaining: stop and give undefined instead of throwing (Java:
// Optional.map, Python: nothing built in)
console.log(doc.description?.toUpperCase());

// ?? nullish coalescing: default ONLY for null/undefined. `||` also replaces 0, "" and
// false: a classic bug.
const gap = 0;
console.log(gap || 16, gap ?? 16); // 16 (wrong), 0 (right)

// strictNullChecks (part of `strict`): null/undefined are NOT members of every type,
// unlike Java references.
function titleLength(d: Doc | undefined): number {
  if (!d) return 0; // narrowing removes undefined
  return d.title.length;
}
console.log(titleLength(doc), titleLength(undefined));

// JSON drops undefined but keeps null: this matters for every HTTP body we send.
console.log(JSON.stringify({ a: undefined, b: null })); // {"b":null}
```

WHAT IT PRINTED

```
undefined null
undefined
```

```
16 0
8 0
{"b":null}
```

The `||` trap is the third output line: `0 || 16` is `16` because `0` is falsy. A layout gap of zero silently becomes sixteen. Use `??`.

BREAK IT • 06-nothing.broken.ts

```
// exactOptionalPropertyTypes: `description?: string` means "absent or a string",
// NOT "may be set to undefined".
type Doc = { title: string; description?: string };
const d: Doc = { title: "Checkout", description: undefined };
console.log(d);
```

Fails with TS2375. `exactOptionalPropertyTypes` : an optional field may be *absent*; it may not be *present and undefined*. The difference is real: `JSON.stringify` drops one and a database `UPDATE` built from object keys would overwrite with the other.

7 • Modules and the .ts rule

A file is a module; `export` is its public surface. Because we run TypeScript straight on Node with no build step, two rules apply that you will not see in older tutorials: spell the real file name in relative imports, extension included, and import types with `import type` so nothing is left dangling after erasure.

TYPESCRIPT	JAVA	PYTHON
file = module, explicit <code>export</code>	class = file, <code>public</code> , packages	file = module, everything importable
<code>node_modules</code> + package exports	classpath / module path	<code>sys.path</code> / site-packages
pnpm workspace	Maven/Gradle multi-module	uv/poetry workspace

RUN IT • node 07-modules.ts

```
// One file = one module (like Python; unlike Java's one-class-per-file + package).

import { makeOrg } from "./07-lib.ts"; // a VALUE import: exists at runtime
import type { Org } from "./07-lib.ts"; // a TYPE import: deleted before the code runs
runs
```

```
// Two repo rules that come from running TypeScript directly on Node (no build step):
// 1. relative imports spell the real file name, extension included: "./07-lib.ts"
// 2. types are imported with `import type`, because after erasure there is no `Org` to
import

const org: Org = makeOrg("Acme");
console.log(org);

// Packages are imported by NAME, resolved through node_modules and the package's
"exports" field:
//   import { HealthResponse } from "@noon/contracts";   ->
packages/contracts/package.json "exports"
// There is no classpath and no PYTHONPATH. tsconfig "paths" aliases do NOT work at
runtime.

// Top-level await works in modules, and import.meta replaces __file__-style tricks.
console.log(import.meta.filename.endsWith("07-modules.ts"));
```

WHAT IT PRINTED

```
{ id: 'org_acme', name: 'Acme' }
true
```

BREAK IT • 07-modules.broken.ts

```
// Java/Python habit: leave the extension off. Node looks for a file literally named
"./07-lib".
// @ts-expect-error - the compiler objects too; silenced here so you can see the
RUNTIME failure
import { makeOrg } from "./07-lib";
console.log(makeOrg("Acme"));
```

Fails with ERR_MODULE_NOT_FOUND. Leaving the extension off is the Java/Python reflex. Node looks for a file literally named `./07-lib` and finds none.

8 • Promises and the single thread

Node runs your JavaScript on **one thread**. I/O is non-blocking: `await` hands the thread back to the event loop until the result arrives. Consequences you will feel in this project: no data races on in-memory state (the sync room's document needs no lock), and one slow synchronous function freezes every request and every socket at once. A blocked event loop is also how a server loses a lease without noticing; epic 7 is built around that.

TYPESCRIPT	JAVA	PYTHON
<code>Promise<T></code> , <code>async/await</code>	<code>CompletableFuture<T></code> , virtual threads	<code>awaitables</code> , <code>asyncio</code>
<code>Promise.all([a, b])</code>	<code>allOf</code> / structured concurrency	<code>asyncio.gather</code>
one thread + event loop	many threads, locks	one thread + event loop (GIL aside)

RUN IT · `node 08-async.ts`

```
// Java: threads + blocking calls. Python: asyncio. Node: ONE thread running your code,
// an event loop,
// and non-blocking I/O. Nothing runs in parallel with your JavaScript, so there are no
// data races on
// in-memory state, and one slow synchronous function freezes EVERYTHING (every request,
// every socket).

const sleep = (ms: number): Promise<void> => new Promise((resolve) => setTimeout(resolve,
ms));

// A Promise<T> = Java's CompletableFuture<T> = Python's awaitable. `async` makes a
function return one.
async function fetchOrg(id: string): Promise<{ id: string }> {
  await sleep(50); // gives the thread back to the event loop while "waiting"
  return { id };
}

// Sequential vs concurrent: the most common performance mistake.
let t = Date.now();
await fetchOrg("a");
await fetchOrg("b");
console.log("one after another:", Date.now() - t, "ms"); // ~100

t = Date.now();
await Promise.all([fetchOrg("a"), fetchOrg("b")]); // both waits overlap
console.log("concurrently:      ", Date.now() - t, "ms"); // ~50

// Errors: a rejected promise is an exception at the `await`.
async function mayFail(): Promise<never> {
  throw new Error("db is down");
}
try {
  await mayFail();
} catch (e) {
  console.log("caught:", e instanceof Error ? e.message : e); // `e` is unknown: narrow
  before use
}

// Ordering: sync code first, then microtasks (promise callbacks), then timers.
setTimeout(() => console.log("3 timer"), 0);
void Promise.resolve().then(() => console.log("2 microtask"));
```

```
console.log("1 sync");

// The classic bug: forgetting `await`. The call starts, nobody waits, errors vanish.
// Our ESLint rule `no-floating-promises` turns that into a lint error; `void` above says
"on purpose".
await sleep(10);
```

WHAT IT PRINTED

```
one after another: 103 ms
concurrently:      51 ms
caught: db is down
1 sync
2 microtask
3 timer
```

BREAK IT • 08-async.broken.ts

```
// Forgetting `await` gives you the Promise, not the value. The type system catches
it.
async function fetchOrg(): Promise<{ id: string }> {
  return { id: "org_1" };
}
const org = fetchOrg(); // missing await
console.log(org.id); // Property 'id' does not exist on type 'Promise<{ id: string;
}>'.
```

Fails with TS2339. Forget `await` and you hold the promise, not the value. The compiler catches the property access; our lint rule `no-floating-promises` catches the case where you ignore the result entirely.

9 • Reading this repo's config

Three files do the job of `pom.xml` / `pyproject.toml`. Open each one; they are short.

FILE	JOB	CLOSEST THING YOU KNOW
package.json	name, dependencies, "type": "module", and "exports": the one door into a package	pom.xml · pyproject.toml
pnpm-workspace.yaml	says apps/* and packages/* are one repo of many packages, linked by symlink	Maven multi-module · uv workspace
pnpm-lock.yaml	exact resolved versions; committed	poetry.lock
tsconfig.json	how strict the <i>checker</i> is. It has no effect on how code <i>runs</i>	javac -Xlint · mypy config

The tsconfig.json flags you met above, all on in this repo:

FLAG	WHAT IT BOUGHT YOU	SECTION
strict	null / undefined are not in every type; no implicit any	6
noUncheckedIndexedAccess	xs[0] may be undefined	4
exactOptionalPropertyTypes	absent is not the same as undefined	6
verbatimModuleSyntax	forces import type for types	7
erasableSyntaxOnly	bans enum and friends, which Node cannot strip	5
noEmit	the compiler only checks; Node runs the source	5

One real story from setting up this repo. The newest TypeScript (7) crashed our lint layer, because the lint plugin only supports versions below 6.1. We pinned 6.0. In this ecosystem the compiler, the linter and the runtime are three separate projects on three release schedules; in Java they ship together. Expect version pinning to be part of the job.

Recall first

Look away. Say the eight ideas out loud, and for each one the thing Java or Python does differently. Then check.

▼ Check yourself

Values and inference (one `number`) · structural typing · unions and narrowing · functions and inferred generics · types vanish at runtime, so validate at boundaries · `null` vs `undefined`, `?.` and `??` · modules, `.ts` extension, `import type` · promises on a single thread.

Then test it

1. A request body is cast with `as CreateOrg`, but the client sent `{"name": 42}`. What happens?

The compiler reports an error

Nothing; it fails later, somewhere else

The cast throws a `TypeError`

Node rejects the request with 400

Types are erased; `as` checks nothing. Only runtime code (our Zod contracts) can reject it, which is why every boundary parses.

2. `class Dog { name = "" }` and `function greet(x: { name: string })`. Does `greet({ name: "Rex" })` compile?

Yes: the shape matches

Only with an `implements` clause

No: the object is not a `Dog`

Only after a cast

Structural typing. Names and declarations do not matter; shapes do.

3. Why does this repo use `"queued" | "running"` instead of an `enum` ?

Node strips types; enums need generated code

Unions allow more than two values

Enums are slower at runtime

Enums cannot be used in a switch

We run source directly. Anything that is not pure type syntax cannot be erased, so Node refuses it.

4. `const gap = props.gap || 16` and the designer set gap to 0. What is `gap` ?

undefined

16

A compile error

0

`||` falls through on every falsy value, including 0 and `""`. `??` falls through only on null and undefined.

5. Two independent 50 ms database calls, each awaited on its own line. Total wait?

About 50 ms

It depends on the thread pool size

About 100 ms

About 0 ms; await does not block

Each `await` finishes before the next line starts. `Promise.all` overlaps the waits.

6. A handler runs a 3-second synchronous loop. What do other connected clients experience?

Nothing; other requests use other threads

Node kills the handler after a timeout

Only that client's socket stalls

Everything stalls for 3 seconds

One thread runs all your JavaScript. While it is busy, no timer fires, no socket is read, no heartbeat is answered.

7. You add a fifth op type to the union and forget one `switch` . When do you find out?

At runtime, with an exception

At compile time, on the `never` line

Never; the default case hides it

Only if a test covers it

The exhaustiveness check turns "I forgot a case" into a type error in every switch that matters.

Next: Lesson 1, written after epic 1 is built. How the typed API, the Postgres layer and tenant isolation were written, line by line, and drills where you fix a planted bug.